2026 학년도 모바일공학전공 하반기 공모전

[트랙 2] 임베디드 환경 On-device AI 최적화 챌린지

예선 결과 보고서 — 자유주제

1. 팀 정보

구분	내용
팀명	팀명 기입
선택 주제	자유주제 — 자유 모델 최적화
대상 모델명	OpenAI CLIP ViT-B/16 이미지 인코더 (open_clip `ViT-B-16`, pretrained=`openai`)
팀원 (성명 / 학번 / 역할)	성명 / 학번 / 역할
	성명 / 학번 / 역할
	성명 / 학번 / 역할
대표자 연락처	연락처 기입
제출일	2026 년 월 일

2. 결과 요약

항목	Baseline	최적화 후	변화율
모델 크기 (MB)	344.9	89.8	−74.0%
파라미터 수 (M)	86.19	86.20 (분리용 행렬 포함)	≈0% (수는 그대로, 32→8 비트)
연산량 (GFLOPs)	35.13	35.13	0% (INT8 연산으로 수행)
추론 Latency (ms)	58.06 (p50)	25.51 (p50)	−56.1% (2.28 배)
Throughput (FPS)	17.2	39.2	+128%
피크 메모리 사용량 (MB)	516	232	−55.0%
정확도	COCO-confirm R@1 41.80%	40.85%	−0.96%p [95% 범위 −1.56, −0.35]

요약

OpenAI CLIP ViT-B/16 이미지 인코더(사진 검색의 색인 단계)를 학습 없는 정적 INT8 로 양자화했다. 원인 분석으로 찾은 **블록 5~7 fc2 입력의 이상치 32 채널만 FP32 작은 행렬곱으로 분리(이상치 채널 분리, SPL)** 하고 나머지는 INT8 로 두었다.

PC CPU(ONNX Runtime, 4 스레드) 기준 **지연 2.28 배 단축, 크기 −74%, 메모리 −55%**이고, 자체 COCO 텍스트→이미지 검색 **R@1 손실은 −0.96%p** 로 결과를 보기 전에 봉인한 판정용 분할에서 사전 기준(−2%p)

을 통과했다. CIFAR-100 제로샷은 68.01→68.66%. 사진 1,000 장 색인은 71.6 초→31.5 초.

같은 전략을 Hailo-10H(Dataflow Compiler 5.4.0)에 이식해 공식 설정과 같은 13 컨텍스트로 컴파일했으나, 판정용 분할 R@1 은 −3.03%p(공식 설정 −3.16%p)로 기준을 넘지 못했다. 원인 분석과 본선 대응은 6.6·7 장.

재현: 다른 PC(새 환경)에서 보고서의 정확도 8 개 값이 모두 그대로 재현됐다(6.3).

3. 대상 모델 및 선정 사유

3.1 대상 모델

구분	내용
모델명 / 버전	CLIP ViT-B/16 이미지 인코더 (open_clip `ViT-B-16`, QuickGELU, 입력 224×224, 출력 512 차원)
모델 유형 (비전 / 언어 / 멀티모달 등)	멀티모달(이미지-텍스트 대조 학습) 모델의 이미지 인코더 = 비전 트랜스포머(12 블록, 폭 768)
저장소 URL	https://github.com/mlfoundations/open_clip (원 모델: https://github.com/openai/CLIP)
커밋 해시 또는 릴리스 태그	open_clip_torch 3.3.0 (PyPI 릴리스) / Hailo 경로: Hailo Model Zoo 5.4.0 공식 ONNX
사전학습 가중치 출처	OpenAI 공개 CLIP 가중치 (open_clip `pretrained=openai` 로 자동 다운로드). 학습·미세조정 없음
라이선스	OpenAI CLIP: MIT, open_clip: MIT 제출 전 각 저장소 LICENSE 확인

3.2 선정 사유

한 줄 요약: 실제 사진 검색 앱에서 쓰이는 공개 모델이고, Hailo-10H 공식 지원 모델이라 제조사 기본 양자화와 같은 평가로 직접 비교할 수 있으며, 그 기본 양자화에서도 손실이 큰 모델이라 개선 여지가 분명하다.

왜 이 모델인가

- **실사용:** 오픈소스 사진 관리 앱 Immich 의 문장 검색이 CLIP 계열 모델을 쓰며, 선택 가능한 모델에 `ViT-B-16__openai` 가 있다 [1].
- **Hailo 공식 지원 = 비교 기준 확보:** Hailo Model Zoo 의 Hailo-10H 공개 모델 표에 `clip_vit_b_16_image_encoder` 가 있다(CIFAR-100 제로샷 top-1 FP 68.6 → 하드웨어 64.6) [2]. 같은 평가에서 제조사 기본 설정 대비 개선을 보일 수 있다.
- **개선 여지:** 같은 표에서 ViT-B/16 의 양자화 손실(−4.0%p)은 ViT-B/32(−0.8%p)보다 크다. B/16 은 FP 정확도가 더 높다(68.6 vs 65.3) [2].
- **기술적 난이도:** 트랜스포머의 활성값 이상치는 8 비트 양자화를 어렵게 하는 알려진 문제다 [3][6]. 실제로 ONNX Runtime 기본 정적 INT8 에서 R@1 이 0.19%로 붕괴했다(5.3).

왜 경량화해야 하나

- **비용은 색인에 몰린다:** 사진 검색은 사진마다 이미지 인코더를 한 번(색인), 질의마다 텍스트 인코더를 한 번 돌린다. 실측에서 질의 1 개는 27 ms(p50), 사진 10 만 장과의 비교 계산은 4 ms 인 반면, 색인은 FP32 에서 사진 1 장당 71.6 ms 였다(6.5). 갤러리가 클수록 이미지 인코더가 비용을 지배한다.
- **실시간 처리 가능 여부:** 디코드를 포함한 사진 1 장 처리가 FP32 71.6 ms 에서 INT8 31.5 ms 로 줄어 30 fps(33.3 ms) 안에 들어온다(여유는 작다). 1,000 장 색인 최대 메모리는 508→213 MB.
- **기기 안 처리:** 사진을 서버로 보내지 않아 개인정보를 보호하고 오프라인에서도 동작한다. NPU 로 옮기면 CPU 를 다른 작업에 쓸 수 있다.
- **기존 텍스트 검색과 호환:** 텍스트 인코더는 FP32 그대로 두어 INT8 이미지 임베딩을 기존 CLIP 텍스트 질의로 그대로 검색한다(6.5 의 R@1 이 바로 이 조합의 측정값). 더 작은 다른 모델(예: MobileCLIP [5])로

바꾸면 임베딩 공간이 달라져 색인을 모두 다시 만들어야 한다.

3.3 Hailo-10H 호환성 사전 검토

결론: CPU 폴백 없이 그래프 전체가 Hailo-10H 용 HEF 로 컴파일됐다(DFC 5.4.0, 에뮬레이터로 정확도 검증). 실기기 동작·속도는 본선에서 확인한다. 또한 예선 단계의 기법 선택 자체를 NPU 구동을 전제로 내렸다(아래 마지막 항목).

컴파일 가능성 확인

- **파싱:** Hailo Model Zoo 공식 ONNX 를 시작·끝 노드 지정 없이 `translate_onnx_model` 로 변환했다. 앞뒤를 잘라 CPU 에 남긴 부분이 없다.
- **구조 확인(컴파일된 HAR):** 입력 224×224×3(uint8) → 출력 512 차원. 입력 정규화가 칩 안에 있고 호스트·후처리 층이 없다(공식·우리 구성 모두). 층 구성은 conv 74, matmul 24, 정규화·활성·원소 연산 등 표준 연산뿐이다(`logs/dfc/hef_check.md`).
- **컴파일 결과:** 공식 설정 HEF 79.3 MB·13 컨텍스트, 우리 최종 구성(이상치 갈래 16 비트) 79.6 MB·13 컨텍스트로 공식과 같다. 참고로 7 번 층 전체를 16 비트로 두면 14 컨텍스트가 된다.
- **정밀도 대응:** Hailo-10H 는 INT8 과 16 비트(a16_w16)를 층별로 섞을 수 있어, ORT 에서 FP32 로 남긴 이상치 채널 갈래를 16 비트로 대응했다. 활성 16·가중치 8 비트(a16_w8)는 DFC 내부 오류로 컴파일되지 않아 쓰지 않았다. INT4(가중치 전용)는 ORT 에서 시험했으나 정확도가 회복되지 않아 제외했다(5.3).
- **텍스트 인코더:** 질의마다 한 번이라 호스트 CPU(FP32)에 둔다. HEF 컴파일도 시도했으나 공식 설정의 미세조정 단계가 이 PC 에서 멈춰 중단했다(7.1).
- **본선 기준값:** Hailo 가 공식 HEF 로 잰 값은 배치 1 51.5 fps, 배치 8 158 fps 다 [2]. 우리 구성은 컨텍스트 수가 같아 비슷한 속도를 기대하되, 보드에서 실측해 확정한다. (DFC 프로파일러의 컨텍스트별 추정을 합산한 값은 전환 비용이 빠져 과대 추정이므로 쓰지 않는다.)

실기기 구동을 위해 내린 설계 선택

- **NPU 에서 돌 수 있는 방식만 채택:** CPU 에서는 동적 INT8+fc2 보호(A1-c67-pc)가 더 정확하고 빨랐지만, 실행 중 스케일을 계산하는 방식이라 NPU 에 올릴 수 없어 최종 후보에서 뺐다. 최종 구성은 고정 스케일 정적 INT8 이다(5.3, 7.1).
- **연산 전부를 칩에:** 이상치 처리를 CPU 로 빼지 않고, 등가 변환(채널 재배열+갈래 분할)으로 그래프 안에 넣어 칩 안에서 처리한다. 입력 정규화도 칩 안이라 호스트는 사진 디코드·리사이즈만 한다.
- **속도 비용 최소화:** 7 번 층 전체를 16 비트로 두면 14 컨텍스트로 늘어 컨텍스트 전환이 추가된다. 32 채널 갈래만 16 비트로 두는 분할을 택해 공식과 같은 13 컨텍스트를 유지했다.
- **칩의 혼합 정밀도 활용:** 대부분은 INT8, 이상치 갈래만 16 비트로 두어 Hailo-10H 의 층별 정밀도 지정 기능을 필요한 곳에만 썼다.
- **메모리 여유:** HEF 79.6 MB 로 온보드 8 GB 메모리에 비해 작아, 색인용 임베딩과 다른 작업을 함께 둘 여유가 있다.
- **역할 분담:** 사진마다 도는 이미지 인코더(비용 대부분, 3.2)는 NPU, 질의당 1 회인 텍스트 인코더는 호스트 CPU 에 둔다. 색인은 사진을 몰아서 처리하는 작업이라 배치·비동기 처리로 NPU 를 쉬지 않게 할 수 있다(배치 8 공개값 158 fps [2]). 본선에서 배치·파이프라인 효과와 CPU 단독 대비 CPU 부하를 실측한다(7.2).

4. 실험 환경

구분	사양 및 버전
CPU	Intel Core i9-14900KF (측정은 4 스레드 고정) / 재현 확인: Intel Core i9-12900
GPU (모델 / VRAM)	사용하지 않음 (정확도·속도·Hailo 컴파일 모두 CPU)
RAM	23.5 GB (WSL2 할당)
OS	Windows + WSL2 Ubuntu 22.04 (커널 6.18.33.2-microsoft-standard-WSL2)
Python 버전	3.10.12
딥러닝 프레임워크 / 버전	PyTorch 2.14.1+cpu, open_clip_torch 3.3.0, ONNX 1.23.1 (opset 17), NumPy 2.2.6, Pillow 12.3.0
CUDA / cuDNN 버전	해당 없음 (CPU 전용, `CUDA_VISIBLE_DEVICES=''`)
양자화·최적화 도구 / 버전	ONNX Runtime 1.23.2 (onnxruntime.quantization, QDQ) / Hailo Dataflow Compiler 5.4.0 (TensorFlow 2.19.1, hailo10h)

5. 적용한 최적화·양자화 기법

5.1 기법 요약

적용 기법	적용 대상	주요 설정값	단독 적용 효과
ONNX 변환·상수 폴딩	이미지 인코더 전체	opset 17, 노드 996→489	출력 일치 (최대 오차 1.03e-5, 코사인 1.000000)
정적 INT8 PTQ (가중치 MatMul 한정, A2m)	가중치 MatMul 49 개	MinMax, 텐서별, QDQ, 캘리브 COCO 64 장	R@1 −6.89%p (ORT 기본 정적은 −41.61%p 붕괴)
혼합 정밀도 (민감층 FP32 유지)	블록 6·7 fc2	해당 층만 양자화 제외	R@1 −6.89 → −1.18%p
채널별 가중치 양자화	가중치 MatMul	per-channel	R@1 −1.18 → −0.86%p
이상치 채널 분리 (최종, SPL-b567-k32)	블록 5·6·7 fc2 입력	캘리브에서 |x| 상위 32 채널만 FP32 작은 행렬곱, 나머지 INT8	R@1 −0.96%p, 25.51 ms (2.28 배), 89.8 MB (fc2 통째 FP32 보다 14 MB 작음)
(Hailo) 이상치 갈래 16 비트	블록 7 fc1·GELU·fc2 의 32 채널	a16_w16, 나머지 공식 설정	13 컨텍스트 유지, R@1 −3.03%p (공식 −3.16)

5.2 양자화 상세

구분	내용
양자화 방식 (PTQ / QAT)	PTQ (정적, 학습·미세조정 없음)
정밀도 (INT8 / INT4 / 혼합)	혼합: 가중치 INT8(채널별)·활성 INT8(텐서별, MinMax) + 이상치 32 채널 ×3 블록 FP32 / Hailo: INT8 + 이상치 갈래 a16_w16
캘리브레이션 데이터셋	COCO val2017 중 평가 분할에서 제외한 사진 (`splits/calib_coco_64_s0.txt`). 평가 사진과의 겹침을 코드로 차단 (겹침 0)
캘리브레이션 샘플 수	64 장 (Hailo 사후 분석: 64 장 3 묶음, 합집합 189 장)
양자화 제외 레이어 (있는 경우)	비가중치 연산(어텐션 활성 MatMul 24 개, Softmax, LayerNorm, Add)은 FP32 / 이상치 채널 행렬곱 3 개(블록 5·6·7) FP32

5.3 적용 근거 및 과정

원인을 측정으로 찾고, 찾은 곳만 고치는 순서로 진행했다. 정확도 수치는 CIFAR-100 검증셋(10,000 장) 기준이다(검색 평가는 6 장).

1. **첫 실패:** ORT 기본 정적 INT8(A2)이 2.09%로 붕괴했다. 채널별·Percentile·Entropy·일부 층 제외 등 7 가지 탐침이 모두 1.5~4.8%에 그쳐, 범위 추정 방식이 원인이 아님을 확인했다. Softmax·Add 까지 양자화한 것이 원인으로 보고 가중치 MatMul 49 개로 제한하자(A2m) 53.6%로 회복했다(FP32 68.8%).
2. **어느 층인가(민감도):** 층 종류·깊이·블록별로 한 묶음씩 FP32 로 남겨 회복량을 쟀다. fc2 12 개를 남기면 손실의 100%, 블록별로는 7 번 75%·6 번 49%·5 번 15%가 회복됐다.

3. **왜 그 층인가(활성값):** 블록 5~7 fc2 입력은 최대값이 17/26/42 인데 99.9%의 값이 8 비트 255 칸 중 9~18 칸에 몰려 있다(그림 2). 이상치는 3~4 개 채널에 집중돼 있다.
4. **처방 1, 민감층 보호:** 6·7 번 fc2 를 FP32 로 두고 가중치를 채널별로 양자화하자 69.09%로 FP32 수준이 됐다.
5. **처방 2, 이상치 채널 분리(최종):** 층 전체가 아니라 이상치 32 채널만 FP32 행렬곱으로 떼어 내고 나머지를 INT8 로 두었다. 원본과 등가 변환인지 확인했다(코사인 0.9999999). FP32 로 남는 부분이 작아 모델이 더 작다(89.8 MB vs 103.7 MB).
6. **Hailo 이식:** fc2 만 16 비트로 나누면 실패했다(fc1+GELU 출력이 먼저 8 비트로 깎임). 그래서 fc1 부터 갈래를 나눠 13 컨텍스트를 유지하며 select 분할에서 기준을 통과했다. 판정용 분할 결과는 6.6 에서 다룬다.
7. **선택 규칙:** 후보는 NPU 에서 실행 가능한 정적 구성으로 한정했다. 동적 INT8+fc2 보호(A1-c67-pc)는 CPU 에서 더 정확하고 빠르지만(−0.57%p, 24.05 ms) 실행 중 범위를 계산하는 방식이라 NPU 이식 전제에 맞지 않아 비교 기준으로만 남겼다.

기각한 시도 (근거 포함)

- **INT4 가중치 전용:** CPU 속도 이득이 없고(60 ms), fc2 를 보호해도 정확도가 회복되지 않았다(가중치 오차라는 다른 실패 양상).
- **QOperator 형식:** QDQ 와 속도 차이가 없어 QDQ 를 유지했다.
- **CIFAR 사진으로 캘리브:** CIFAR 정확도는 올랐지만 실제 사진 검색 R@1 이 23.7%로 무너졌다. 같은 도메인 효과였고, 이 결과로 평가 축을 검색으로 바꿨다(6.2).
- **테스트 시 레지스터[4]:** 원인 설명(이상치 = 레지스터 뉴런)은 확인했으나, 정확도 유지와 범위 축소를 동시에 만족하는 설정이 없었다(−2.5~−16%p).
- **블록 제거(ReplaceMe):** 블록 1 개 제거도 R@1 −1.26%p [−2.22]로 기준 미달.
- **입력 해상도 축소:** 192 에서 −1.14%p 로 경계였다. 별도 선택지로만 기록했다.

6. 벤치마크 설계 및 측정 결과

6.1 선정한 벤치마크

구분	내용
데이터셋 / 벤치마크명	(주) COCO val2017 텍스트→이미지 검색, 자체 분할 / (보조) CIFAR-100 제로샷 분류
공개 여부 및 출처 URL	공개. COCO: https://cocodataset.org (val2017 이미지, captions_val2017) / CIFAR-100: https://www.cs.toronto.edu/~kriz/cifar.html
평가셋 규모 (샘플 수)	COCO val2017 5,000 장 = 캘리브 제외 189 / 선택용 select 2,405 장(캡션 12,035) / 판정용 confirm 2,406 장(캡션 12,033). CIFAR-100: 검증 10,000(train 에서 분리) / test 10,000
평가 지표	Recall@1(주), R@5, R@10 = 캡션으로 검색했을 때 정답 사진이 상위 K 장 안에 있는 비율(이미지별 평균). 차이는 이미지 단위 짝지은 bootstrap 2,000 회 95% 범위. 보조: CIFAR top-1
지표 선정 사유	목표 응용(문장으로 사진 찾기)을 그대로 측정한다. 텍스트 인코더를 FP32 로 고정해 이미지 인코더 양자화의 영향만 분리한다

6.2 벤치마크 선정 사유 및 타당성

- **응용을 직접 잰다:** R@K 는 "검색 결과 상위 K 장 안에 원하는 사진이 있나"다. R@1 을 주지표로, 화면 첫 줄에 해당하는 R@5·R@10 을 함께 보고한다.
- **결과를 보기 전에 규칙을 고정했다:**
  - 분할(제외 189 / select / confirm)을 첫 평가 전에 파일로 고정하고 sha256 을 기록했다. 캘리브 사진은 평가 사진과 겹치지 않게 코드로 막았다.
  - 구성은 select 로만 골랐다. confirm 과 CIFAR test 는 구성을 동결(10/17)한 뒤 단 한 번 열었다(`--i-know-final` 없이는 열리지 않게 코드로 잠금).
  - 차이는 같은 이미지끼리 짝지은 bootstrap 95% 범위로 보고한다. "FP32 와 거의 같다"의 기준(하한 ≥ −2%p)은 갤러리 약 2,400 장의 측정 오차(약 ±1.4%p)를 고려해 정했다. select 결과를 본 뒤 정했기 때문에 판정은 confirm 한 번으로만 했다.
  - 우리 ONNX·텍스트 임베딩이 Hugging Face 공식 CLIP 과 같은지 확인했다(코사인 1.000000).
- **분류만으로는 공정하지 않았다:** 초기에는 CIFAR-100 으로 실험했다. 그런데 CIFAR 사진으로 캘리브한 Hailo 모델이 CIFAR 정확도는 오르면서 실제 사진 검색 R@1 은 23.7%로 무너졌다(FP 40.9%). 그래서 검색을 주지표로 바꾸고 CIFAR 는 보조로 남겼다.

선택하지 않은 대안

- **ImageNet 제로샷 분류:** 분류라 응용(검색)과 다르고 CIFAR 와 같은 한계가 있다.
- **공개 검색 분할(COCO 5K 등):** 캘리브·선택·판정 사진을 분리하기 어려워 자체 분할을 썼다. 그래서 공개 논문 수치와 직접 비교하지 않는다.
- **속도만 보기:** 정확도 조건 없이 빠른 구성(예: 동적 INT8 −1.48%p)은 마진 하한(−2.18)을 넘지 못해 탈락시켰다.

6.3 재현 가능성 확보 방안

- **데이터:** 모두 공개 데이터이고 공식 URL 에서 자동으로 받는다(약 1.2 GB). 비공개 데이터는 없다.
- **명령 하나로 재현:** `bash reproduce.sh` = 다운로드 → 분할 재생성과 제출 분할 sha256 비교 → FP32 export 와 PyTorch 출력 일치 검사 → 양자화 → COCO-confirm·CIFAR test → 속도 → 기대값과 비교 표 출력(9 장).
- **검증 결과:** 원 실험 PC 에서 빈 폴더로 두 번(zip 을 새 폴더에 풀어 한 번 포함), **다른 PC(i9-12900, 새 WSL·새 파이썬 환경)**에서 한 번 실행했다. 세 번 모두 정확도 8 개 값(FP32·최종 구성의 R@1/5/10 과 CIFAR test)이 소수 둘째 자리까지 같았고 신뢰구간도 같았다. 속도 배율은 2.28 배(원 PC)와 2.37 배(다른 PC)였다.
- **소요 시간:** 2~4 시간(다운로드 10~40 분 포함), GPU 불필요. 정확도만 볼 때는 `SKIP_BENCH=1`.
- **제출물:** 코드 zip(약 1.2 MB)과 재현 가이드(`docs/06_재현가이드.md`), 모델 파일(ONNX·HEF)은 공유 링크(8 장).

6.4 측정 프로토콜

측정 조건	설정값
입력 해상도	224×224 (CLIP 표준 전처리: 리사이즈·중앙 크롭·정규화)
배치 크기	1
Warm-up 횟수	20 회
측정 반복 횟수	300 회 × 3 반복 (반복마다 구성 순서를 섞음)
통계 방식 (평균 / p50 / p95)	p50(대표값), p95 병기, 3 반복의 중앙값. 메모리 = 측정 프로세스 최대 상주 메모리(VmHWM)
측정 도구 및 방법	ONNX Runtime 1.23.2 CPUExecutionProvider, intra-op 4 스레드·inter-op 1·순차 실행, 구성마다 새 프로세스, 부하가 낮을 때 측정. 색인 실측은 사진 1,000 장 디코드+전처리+인코더(3 반복 중앙값)

6.5 측정 결과

항목	Baseline	최적화 후	변화율 (%)
정확도 지표 1 (COCO-confirm R@1)	41.80%	40.85%	−0.96%p [−1.56, −0.35]
정확도 지표 2 (CIFAR-100 test top-1)	68.01%	68.66%	+0.65%p
추론 Latency (ms, p50 / p95)	58.06 / 60.02	25.51 / 26.52	−56.1% (2.28 배)
Throughput (FPS, 인코더 단독)	17.2	39.2	+128%
모델 크기 (MB)	344.9	89.8	−74.0%
피크 메모리 (MB)	516	232	−55.0%
정확도 지표 3 (COCO-confirm R@5 / R@10)	68.10% / 78.54%	66.80% / 77.92%	−1.30 / −0.62%p

항목	Baseline	최적화 후	변화율 (%)
사진 1,000 장 색인 (디코드 포함)	71.6 s · 14.0 장/s · 508 MB	31.5 s · 31.7 장/s · 213 MB	2.27 배 · −58%

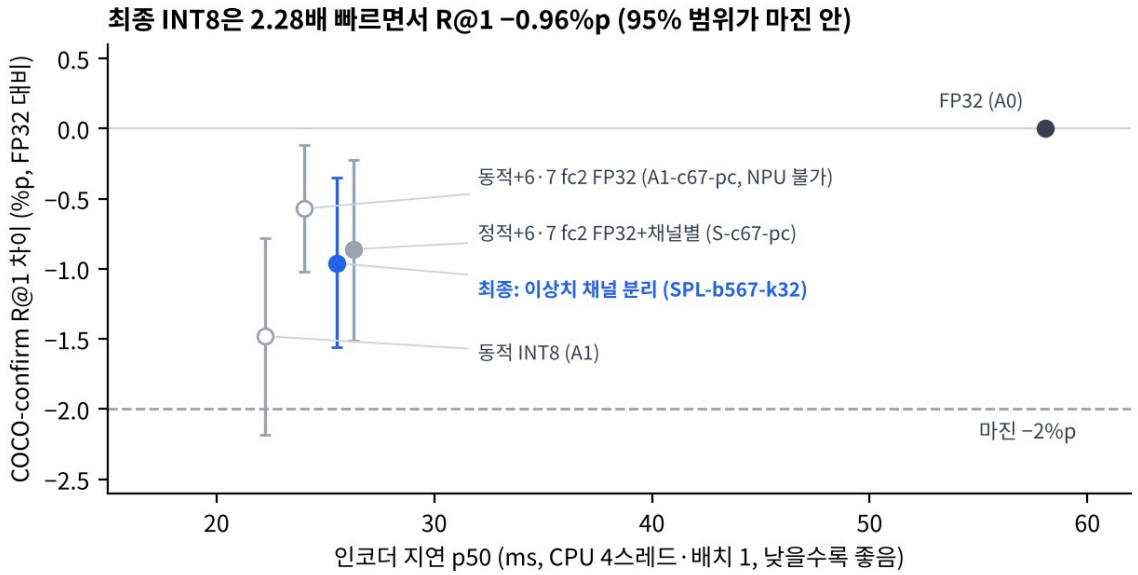


그림 1. ORT 구성별 지연과 검색 정확도 (같은 세션 측정, 95% 범위). 빈 원은 NPU 이식 전제에 맞지 않는 동적 INT8(비교 기준).

Hailo-10H 이식 결과 (같은 COCO-confirm 분할, Hailo FP = 양자화 전 Hailo 그래프)

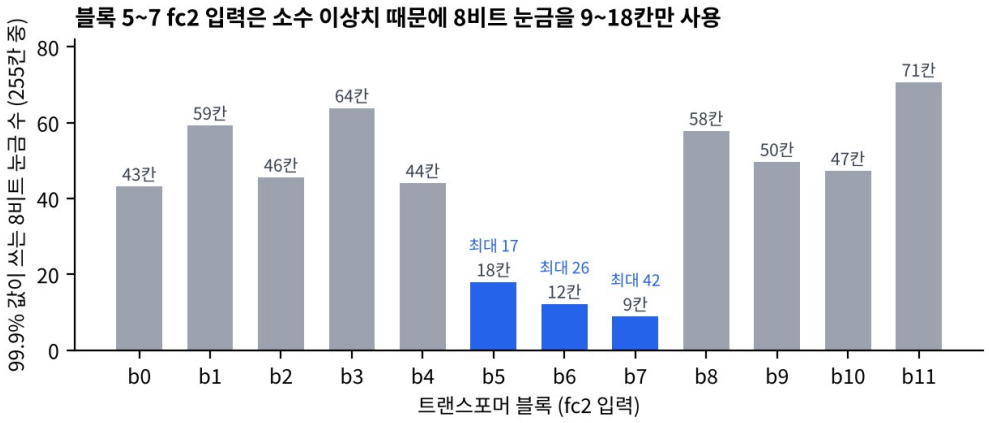
항목 (Hailo-10H, DFC 에뮬레이터)	Hailo FP	공식 설정	우리 최종 (H-splf-b7-k32)
COCO-confirm R@1 (FP 대비)	41.80%	38.65% (−3.16%p)	38.77% (−3.03%p)
COCO-confirm R@5 / R@10	68.10 / 78.54%	64.85 / 76.26%	65.89 / 77.09%
CIFAR-100 test top-1	68.01%	61.96%	65.58%
캘리브 묶음 3 개 confirm R@1 평균 [범위] (사후)	—	−3.16 (1 묶음)	−2.26 [−3.03, −1.41]
컨텍스트 수 / HEF 크기	—	13 / 79.3 MB	13 / 79.6 MB
속도	—	Hailo 발표 51.5 fps (배치 1)	본선 보드 실측 예정

Hailo 정확도는 DFC 에뮬레이터 값이다. 속도는 보드가 없어 측정하지 않았고, 공식 HEF 의 속도는 Hailo 발표값이다 [2].

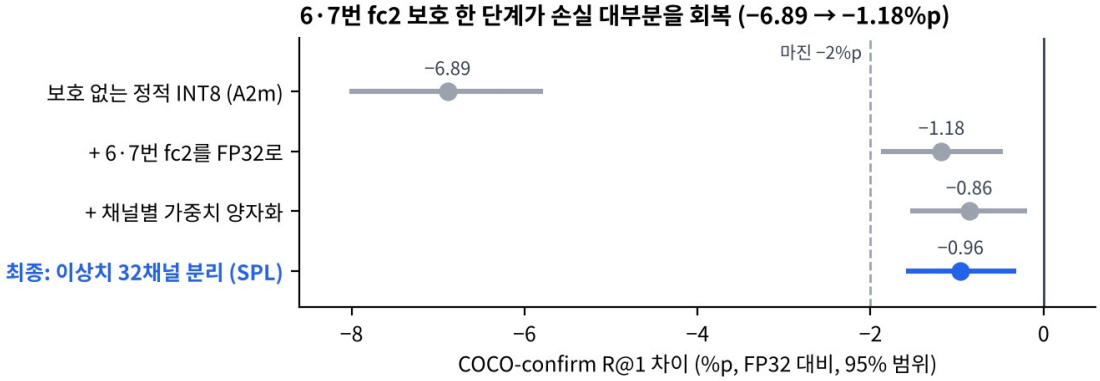
6.6 결과 해석

손실은 어디서 나는가

- 손실은 모델 전체가 아니라 **블록 5~7 fc2 입력의 소수 이상치 채널**에서 생긴다(5.3 의 민감도, 그림 2). 이상치는 사진당 약 5 개 토큰에서만 켜지는 MLP 뉴런이 만들고, 활성이 가장 큰 뉴런 상위 10 개가 모두 우리가 분리한 32 채널 안에 있었다. 이는 비전 트랜스포머의 '레지스터 뉴런' 보고 [4]와 같은 현상이며, 비전 인코더의 8 비트 민감층이 중간 블록 FC2 에 몰린다는 보고 [6]와도 일치한다.



- **단계별 기여(그림 3, 사후 평가):** 보호 없는 정적 INT8 의 손실 −6.89%p 중 대부분이 6·7 번 fc2 보호 한 단계에서 회복된다(−1.18). 최종 구성은 FP32 로 남기는 부분을 32 채널로 줄이고도 같은 수준(−0.96)을 유지한다.



어떤 입력에서 취약했나 (사후 분석)

- 보호 없는 INT8(A2m)은 사진 종류(객체 수·주 객체 크기·범주)와 캡션 종류(숫자·색·위치 표현) **전반에서 고르게 −5~−9%p** 떨어졌다. 특정 입력이 아니라 모델 전체에 걸친 이상치 문제라는 원인 분석과 맞는다.
- 최종 구성에서 일부 묶음의 큰 차이(예: 스포츠 사진 −8.4%p)는 select 분할에서 재현되지 않아 우연으로 판단했다. 여러 묶음을 동시에 보면 하나쯤은 우연히 벗어나기 때문이다.
- Hailo 는 **주 피사체가 화면의 40% 이상인 사진**에서 두 분할 모두 더 떨어졌다(confirm −4.48 / select −2.72%p). 관찰이며 원인은 확인하지 못했다.

ORT 와 Hailo 의 차이

- Hailo 최종 구성은 select −1.10%p 에서 confirm −3.03%p 로 떨어졌다. 사후 분석에서 캘리브 묶음만 바꿔도 confirm 이 −3.03/−2.35/−1.41%p 로 크게 흔들렸고(평균 −2.26), 189 장을 모두 써도 −2.04%p 였다. **64 장 정적 캘리브의 변동**과 select 에서 여러 변형 중 하나를 고른 **선택 효과**가 겹친 것으로 본다. 7 번 층 전체 16 비트(14 컨텍스트)는 3 묶음 평균 −1.73%p 로 더 안정적이었다.
- ORT 는 같은 캘리브로 select −0.78 → confirm −0.96%p 로 안정적이었다. 이상치 채널을 FP32 로 그대로 두기 때문에 캘리브 범위의 영향을 덜 받는 것으로 해석한다(가설).

숫자의 의미

- R@1 −0.96%p 는 질의 100 개 중 약 1 개가 1 등을 놓치는 수준이고, 첫 화면(R@10) 손실은 −0.62%p 다.

CIFAR test +0.65%p 는 표준오차(±0.46%p) 수준이라 향상으로 해석하지 않는다.

7. 한계점 및 엣지 디바이스 이식 계획

7.1 현재 한계점

1. **Hailo 정확도:** 최종 구성이 판정용 분할에서 기준을 넘지 못했다(−3.03%p, 공식 −3.16%p). 64 장 캘리브에 따른 변동이 크다(6.6).
2. **Hailo 속도 미측정:** 보드와 HailoRT 가 없어 실측하지 못했다. DFC 프로파일러의 컨텍스트별 추정은 전환 비용이 빠져 신뢰할 수 없어 쓰지 않았고, 컨텍스트 수(13)만 확인했다.
3. **텍스트 인코더 HEF 미완:** 공식 설정의 미세조정 단계가 이 PC 에서 WSL 전체를 멈춰 중단했다. 현재 텍스트 인코더는 호스트 CPU(FP32)다.
4. **측정 환경:** 속도는 데스크톱 CPU 4 스레드 기준이다. 30 fps 는 여유 약 2 ms 로 겨우 들어온다. 엣지 CPU 에서는 절대값이 다르다.
5. **평가 설계:** 자체 분할이라 공개 수치와 직접 비교할 수 없다. 마진은 select 를 본 뒤 정했다(판정을 confirm 한 번으로 보완).
6. **NPU 전제로 제외한 더 나은 CPU 구성:** 동적 INT8+fc2 보호(A1-c67-pc)는 CPU 에서 더 정확하고 빨랐지만 NPU 에서 실행할 수 없어 제외했다.

7.2 본선 이식 계획

시기	수행 내용	예상 리스크
11/6~11/9	보드·HailoRT 설치, 공식 HEF 와 우리 HEF 구동, 에뮬레이터 임베딩과 일치 확인	런타임 버전 불일치 → DFC 5.4.0 과 맞는 버전 확인
11/10~11/16	예선과 같은 벤치마크·프로토콜로 실측: 배치 1/8 지연·FPS, 메모리·전력, 보드 COCO-confirm 정확도. NPU 활용 지표: CPU 단독 대비 호스트 CPU 부하·전력, HailoRT 도구로 가능한 범위의 NPU 사용률	컨텍스트 전환 비용으로 FPS 가 기대보다 낮을 수 있음
11/17~11/23	Hailo 정확도 개선: 캘리브 사진 확대·여러 묶음 평균으로 선택, 7 번 층 전체 16 비트(평균 −1.73%p)와 비교해 확정	정확도와 컨텍스트 수(속도)의 절충
11/24~11/30	갤러리 검색 시연(보드 색인 + 호스트 텍스트 질의), 색인 파이프라인(사진 디코드와 NPU 추론 겹치기·배치) 적용, 텍스트 인코더 HEF 재시도(성공 시 검색 전 과정 NPU), 최종 보고서 제출(11/30)	시연 안정성, 일정 부족
12/1~12/4	발표 자료, 라이브 시연 리허설, 최종 발표(12/4)	—

8. 제출물 체크리스트

No	제출 항목	파일명 또는 링크	확인
1	결과 보고서 (PDF)	`[트랙 2-예선]_자유_[팀명]_보고서.pdf`	☐
2	베이스라인 모델 파일	`clip_vitb16_image_fp32.onnx` (344.9 MB) — 공유 링크	☐

No	제출 항목	파일명 또는 링크	확인
3	최적화 모델 파일	SPL-b567-k32.onnx (89.8 MB, ORT) / clip_vit_b_16_image_encoder.splf_b7_k32.hailo10h.hef (79.6 MB) — 공유 링크	☐
4	모델 포맷 및 버전 정보 명시	ONNX opset 17 · ONNX Runtime 1.23.2 (QDQ) / HEF: Hailo DFC 5.4.0, hailo10h	☐
5	벤치마크 실행 스크립트	reproduce.sh, src/coco_retrieval.py, src/evaluate.py, src/benchmark.py, src/index_bench.py	☐
6	의존성 목록 (requirements 등)	requirements.lock.txt (버전 고정), requirements.txt	☐
7	실행 방법 README	README.md, docs/06_재현가이드.md	☐
8	데이터셋 경로 또는 다운로드 방법	src/download_data.py 가 공식 URL 에서 자동 다운로드 (COCO val2017·captions_val2017, CIFAR-100)	☐

9. 재현 실행 절차

9.1 환경 구성

Windows 10/11 + WSL2 Ubuntu 22.04, 메모리 16 GB 이상 권장, GPU 불필요.

```
# PowerShell(관리자): WSL 설치 후 재부팅
wsl --install -d Ubuntu-22.04
# Ubuntu: 코드 풀기와 파이썬 환경
sudo apt update && sudo apt install -y python3.10-venv unzip
cd ~ && unzip /mnt/c/Users/<사용자>/Downloads/halio_repro_<날짜>.zip && cd ~/halio_project
python3 -m venv ~/venvs/halio && source ~/venvs/halio/bin/activate
pip install --upgrade pip
pip install -r requirements.lock.txt --extra-index-url
https://download.pytorch.org/whl/cpu
export HALIO_DATA_DIR=/mnt/d/halio_data      # 데이터·모델 저장 위치 (선택)
SELFTEST=1 bash run_pilot.sh                 # 2~3 분 점검 (가짜 데이터)
```

9.2 벤치마크 실행 명령

```
# 전체: 베이스라인(FP32)과 최적화 모델 모두, 정확도 + 속도 + 기대값 비교
bash reproduce.sh
SKIP_BENCH=1 bash reproduce.sh               # 정확도만
# 개별 명령 (M=$(python -m src.common models-dir))
# 베이스라인 FP32
python -m src.coco_retrieval eval --model $M/clip_vitb16_image_fp32.onnx --code RP-A0 --
split confirm --i-know-final
python -m src.evaluate --model $M/clip_vitb16_image_fp32.onnx --code RP-A0 --quant-mode
fp32 --eval-set test --i-know-final --threads 4
# 최적화 모델 (SPL-b567-k32)
python -m src.coco_retrieval eval --model $M/imp/SPL-b567-k32.onnx --code RP-SPL-b567-k32
--split confirm --i-know-final
python -m src.evaluate --model $M/imp/SPL-b567-k32.onnx --code RP-SPL-b567-k32 --quant-
mode static --eval-set test --i-know-final --threads 4
```

```
# 속도·메모리, 색인 실측
python -m src.benchmark --threads 4 --out-csv results/latency_repro.csv --models RP-
A0=$M/clip_vitb16_image_fp32.onnx RP-SPL-b567-k32=$M/imp/SPL-b567-k32.onnx
python -m src.index_bench run
```

9.3 기대 출력

`reproduce.sh` 마지막에 아래 표가 나온다(다른 PC 에서 실제로 나온 출력).

```
구성      항목          제출값  재현값 판정
A0             COCO-confirm R@1        41.79%   41.79%  일치
A0             CIFAR-100 test top-1    68.01%   68.01%  일치
SPL-b567-k32   COCO-confirm R@1        40.84%   40.84%  일치
SPL-b567-k32   CIFAR-100 test top-1    68.66%   68.66%  일치   (R@5·R@10 포함 8 행)
RP-SPL-b567-k32    34.81 ms   FP32 대비 2.37 배  (속도는 CPU 마다 다름)
종합: 정확도 재현 성공
```

- **대응:** CIFAR test → 6.5 정확도 지표 2. 속도 배율 → 6.5 Latency(원 PC 2.28 배). `results/coco_ret_confirm_repro.md` 의 신뢰구간 표 → 6.5 정확도 지표 1·3 과 그림 1.
- **R@1 표기 차이:** 재현 표의 R@1(41.79%)은 캡션 단위 평균, 보고서 표(41.80%)는 이미지별 성적의 평균이다. 같은 결과를 두 방식으로 계산한 것이다.
- **판정 기준:** 정확도는 제출값과 ±0.2%p 안이면 일치. 속도는 절대값이 아니라 FP32 대비 배율로 본다.

9.4 재현 시 유의사항

- **최초 실행 2~4 시간:** 데이터 약 1.2 GB 다운로드(10~40 분), CLIP 가중치는 open_clip 이 자동으로 받는다. 끊겨도 다시 실행하면 끝난 단계는 건너뛴다.
- **저장 공간:** 데이터·모델 약 4~5 GB(`HALIO_DATA_DIR`), 파이썬 환경 약 2 GB.
- **torch 는 CPU 판(2.14.1+cpu):** `--extra-index-url` 을 빼면 GPU 판(약 6 GB)을 받거나 설치에 실패한다.
- **속도 측정:** 다른 프로그램을 끄고 Windows 절전을 해제한다. 절대값은 CPU 마다 다르다.
- **공유한 모델 파일 사용:** `$M/`과 `$M/imp/`에 같은 이름으로 두면 양자화 단계를 건너뛴다.
- **Hailo HEF:** 재컴파일에는 Hailo DFC 5.4.0 이 필요하다(Hailo 개발자 사이트 로그인, 재배포 불가). ORT 수치 재현에는 필요 없다.
- **자주 막히는 곳:** WSL 사용자 이름은 영어 소문자로 시작해야 한다. 윈도우 경로(`C:\...`)는 `/mnt/c/...`로 쓴다. 압축이 풀린 폴더 이름은 `halio_project` 다.

참고 문헌

[1] Immich 문서, Searching. https://docs.immich.app/features/searching

[2] Hailo Model Zoo, Hailo-10H Zero-shot Classification. https://github.com/hailo-ai/hailo_model_zoo/blob/master/docs/public_models/HAILO10H/HAILO10H_zero_shot_classification.rst

[3] Y. Bondarenko, M. Nagel, T. Blankevoort, Quantizable Transformers: Removing Outliers by Helping Attention Heads Do Nothing, NeurIPS 2023. https://arxiv.org/abs/2306.12929

[4] N. Jiang, A. Dravid, A. Efros, Y. Gandelsman, Vision Transformers Don't Need Trained Registers, NeurIPS 2025. https://arxiv.org/abs/2506.08010

[5] MobileCLIP: Fast Image-Text Models through Multi-Modal Reinforced Training, CVPR 2024.

https://arxiv.org/abs/2311.17049
[6] Activation Quantization of Vision Encoders Needs Prefixing Registers, ICML 2026 Workshop (AdaptFM). https://arxiv.org/abs/2510.04547
[7] A. Radford et al., Learning Transferable Visual Models From Natural Language Supervision (CLIP), ICML 2021. https://arxiv.org/abs/2103.00020